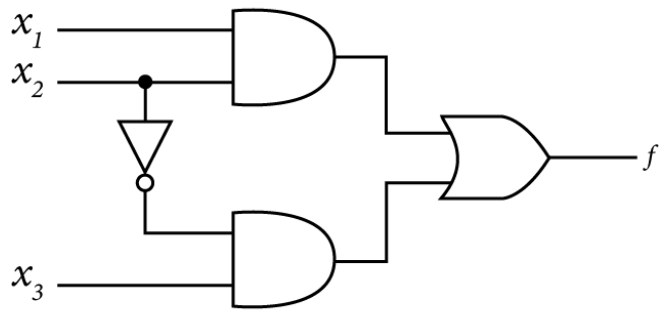
BRAC UNIVERSITY
CSE460: VLSI Design Lab

Experiment 1: Introduction to Verilog HDL and familiarization with Altera Quartus as simulation tool for Verilog HDL codes.

Verilog was originally intended for simulation and verification of digital circuits. Subsequently, with the addition of synthesis capability, Verilog has also become popular for use in design entry in CAD systems. The CAD tools are used to synthesize the Verilog code into a hardware implementation of the described circuit.
Verilog allows the designer to represent circuits in two fundamentally different ways. One possibility is to use Verilog constructs that represent simple circuit elements such as logic gates or even transistors. A larger circuit is defined by writing code that connects such elements together. This is referred to as the structural representation of logic circuits. The second possibility is to describe a circuit by using logic expressions and programming constructs that define the behavior of the circuit but not its actual structure in terms of gates. This is called the behavioral representation.

Example 1: Construct the following circuit using Verilog HDL and verify the output through a timing diagram.



x_3	x_2	x_1	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

a. Structural Representation

```
module example1(f, x1, x2, x3);
      input x1, x2, x3;
      output f;
      wire g, y, h;
      and(g, x1, x2);
      not(y, x2);
      and(h, y, x3);
      or(f, g, h);
endmodule
```

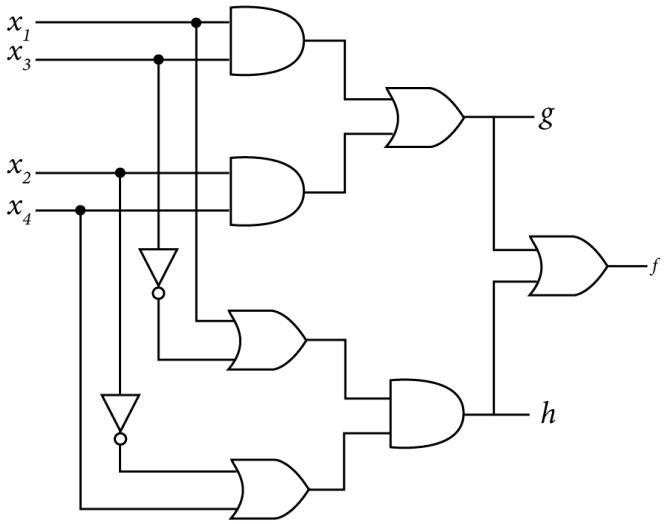
b. Behavioral Representation 1

```
module example1(f, x1, x2, x3);
      input x1, x2, x3;
      output f;
      assign f = (x1 & x2) | (~x2 & x3);
endmodule
```

c. Behavioral Representation 2

```
module example1 (f, x1, x2, x3);
      input x1, x2, x3;
      output f;
      reg f;
      always @(x1 or x2 or x3)
      if (x2 == 1)
      f = x1;
      else
      f = x3;
endmodule
```

Example 2: Construct the following circuit using Verilog HDL and verify the output through a timing diagram.



x_4	x_3	x_2	x_1	f
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

a. Structural Representation

```
module example2(f, x1, x2, x3, x4);
      input x1, x2, x3, x4;
      output f;
      and(w1, x1, x3);
      and(w2, x2, x4);
      or(g, w1, w2);
      or(w3, x1, ~x3);
      or(w4, x4, ~x2);
      and(h, w3, w4);
      or(f, g, h);
endmodule
```

b. Behavioral Representation

```
module example2(f, x1, x2, x3, x4);
      input x1, x2, x3, x4;
      output f;
      assign f = ((x1 & x3)|(x2 & x4))|((x1|~x3)&(~x2|x4));
endmodule
```

Additional Practice:

1. **a.** Design a 1-bit full adder.

   b. Using the 1-bit full adder module, design a 4-bit full adder using Verilog HDL.
2. Design a 4 to 1 MUX using Verilog HDL and verify using a timing diagram
3. Design a 4 to 2 decoder using Verilog HDL and verify using a timing diagram.
4. Design a bit adder-subtractor using Verilog HDL and verify using a timing diagram.